ΟΙ ΟΡΓΑΝΩΜΕΝΕΣ ΔΟΜΕΣ ΔΕΔΟΜΕΝΩΝ ΣΕ ΜΟΡΦΗ ΑΡΧΕΙΟΥ ΣΤΗΝ PYTHON

Μ.Ι.Σ. 2025-2026

Εκτός από τους 3 βασικούς τύπους δεδομένων,

- **αριθμοί,**
- **λογικοί** (boolean),
- **συμβολοσειρές** (strings),

και τους 3 σύνθετους τύπους δεδομένων, σε μορφή πινάκων,

- **λίστες** (lists)
- **πλειάδες** (tuples)
- **λεξικά** (dictionaries)

η γλώσσα PYTHON διαθέτει και την δυνατότητα χειρισμού εξωτερικών δομών δεδομένων

- **αρχεία** (files)

οι οποίες μπορούν να παραμένουν μόνιμα στον Η/Υ (να παραμένουν δηλαδή, αποθηκευμένες μόνιμα στον σκληρό δίσκο και να προσπελαύνονται κατά περίπτωση, με ένα πρόγραμμα) σε αντίθεση με τους πίνακες που είναι ενεργοί –μόνο- όταν τρέχει ένα πρόγραμμα.

Η δυνατότητα χειρισμού ενός αρχείου είναι μια παλαιά και γνωστή διαδικασία για πολλές γλώσσες προγραμματισμού.

Μ.Η.Σ. 2025-2026

Τα αρχεία αυτά μπορούν να έχουν την μορφή,

```
ΔΗΜΗΤΡΙΟΥ          20/ 6/1955          ΤΡΟΙΖΗΝΙΑΣ 14
ΙΩΑΝΝΗ             01/12/1990           ΜΕΘΑΝΑ 14
ΣΩΤΗΡΙΟΥ           27/ 8/1973           ΑΘΗΝΑ
ΓΡΗΓΟΡΙΟΥ          01/12/1990           ΠΟΡΟΣ
ΦΡΑΓΚΟΥΛΗ          18/ 2/1959           ΑΣΚΕΛΙ ΠΟΡΟΥ
ΑΝΑΣΤΑΣΙΟΥ         17/10/1954           ΓΚΟΥΡΑ 89
ΝΙΚΟΛΑΟΥ           17/ 2/1986           ΓΑΛΑΤΑΣ
ΙΩΑΝΝΗ             12/10/1945           ΚΑΛΛΟΝΗ
ΓΕΩΡΓΙΟΥ           01/12/1990           ΑΝΩ ΦΑΝΑΡΙ 12
ΓΕΩΡΓΙΑΣ           21/12/1948           ΠΟΡΟΣ
ΝΙΚΟΥ              01/12/1990           ΑΘΗΝΑ
ΚΩΝΣΤΑΝΤΙΝ          1/ 1/1987           ΚΑΛΛΟΝΗ ΤΡΟΙΖΗΝΙΑΣ
ΑΝΔΡΕΑ             01/12/1990           ΚΑΛΛΟΝΗ 67
ΑΝΔΡΕΑ             01/12/1990           ΠΑΤΡΑ
ΠΑΝΑΓΙΩΤΗ          30/ 1/1948           ΠΟΡΟΣ
ΓΕΩΡΓΙΟΥ           20/11/1990           ΓΑΛΑΝΙΑ 34
ΑΘΑΝΑΣΙΟΥ          03/09/2000           ΔΡΥΟΠΗΣ 6
ΓΕΩΡΓΙΟΣ           26/11/1970           ΓΕΦΥΡΑ ΓΑΛΑΤΑ
ΝΙΚΟΛΑΟΥ           19/ 6/1934           ΓΑΛΑΤΑΣ
ΠΕΤΡΟΥ             20/10/1994           ΠΟΡΟΣ
ΔΗΜΗΤΡΙΟΥ          01/12/1990           ΛΑΡΙΣΑ
ΕΛΕΝΗΣ             29/ 8/1991           ΓΑΛΑΤΑΣ
ΚΩΝΣΤΑΝΤΙΝ          3/ 2/1952           ΓΑΛΑΤΑΣ
ΓΕΩΡΓΙΟΥ           21/12/2000           ΠΟΡΟΣ
```

γίνεται αντιληπτό ότι με την μορφή αυτή μπορούμε να γράψουμε σύνθετα προγράμματα τα οποία να αποθηκεύουν, να διορθώνουν, να αναζητούν και να ανακτούν όγκους δεδομένων.

Η μορφή των αρχείων αυτών, μπορεί να χαρακτηριστεί **σαν μια (σχετικά) σύνθετη δομή δεδομένων**.

Μ.Ι.Σ. 2025-2026

Η γλώσσα PYTHON διαθέτει κάποιες ειδικές εντολές (έτοιμες «εργοστασιακές» συναρτήσεις) για να χειριστεί αυτά τα αρχεία,

για να ανοίξουμε ένα αρχείο, γράφουμε

```
my_b_new_arxeio = open('c:/IRIS2000/my-data.txt', 'w')
```

και το οποίο αρχείο μπορούμε να το προσπελάσουμε για

- διάβασμα **'r'**
- γράψιμο **'w'** ΘΑ ΣΒΗΣΕΙ ΤΟ ΠΕΡΙΕΧΟΜΕΝΟ (ΕΑΝ ΕΧΕΙ)
- επέκταση **'a'** διατήρηση των ήδη περιεχομένων (append)
- επέκταση **'+'** διατήρηση των ήδη περιεχομένων και προσθήκη στο τέλος
- επέκταση **'r+'** για διάβασμα και εγγραφή

Τα ορίσματα της εντολής είναι δυο, το όνομα ή η διαδρομή του αρχείου και ο τύπος της χρήσης του.

Εάν δεν δηλώσουμε διαδρομή, η γλώσσα θα αναζητήσει το αρχείο εκεί που είναι εγκαταστημένο το πακέτο της γλώσσας.

Αν δεν δηλώσουμε τι θα κάνουμε με το αρχείο, τότε ανοίγει αυτόματα μόνο για ανάγνωση (δηλαδή σαν 'r').

Μ.Σ. 2025-2026

Προσοχή,

Εάν το αρχείο που ανοίγουμε για γράψιμο υπάρχει ήδη, τότε τα δεδομένα που έχει μέσα του, σβήνονται όλα.

Αν το αρχείο που ανοίγουμε για γράψιμο δεν υπάρχει, τότε το αρχείο δημιουργείται και είναι έτοιμο για να γράψουμε νέα δεδομένα σε αυτό.

Αν το αρχείο που ανοίγουμε για διάβασμα δεν υπάρχει, τότε θα έχουμε μήνυμα λάθους.

Εκτός των βασικών τρόπων μπορούμε να ανοίξουμε το αρχείο και με κάποιους ειδικούς τρόπους προσπέλασης

- διάβασμα με μορφή binary **'rb'**
- γράψιμο σε μορφή binary **'wb'** ΘΑ ΣΒΗΣΕΙ ΕΠΙΣΗΣ ΤΟ ΠΕΡΙΕΧΟΜΕΝΟ (ΕΑΝ ΕΧΕΙ)

Μετά από την επεξεργασία ενός αρχείου μπορούμε να το κλείσουμε με την εντολή,

```
my_b_new_arxeio.close()
```

Η εντολή είναι χρήσιμη εάν σκεφτούμε ότι το ίδιο αρχείο (που ενημερώθηκε ή δεν ενημερώθηκε από το πρόγραμμα μας) θα πρέπει να γίνει **open** από κάποιο άλλο πρόγραμμα και για κάποια άλλη επεξεργασία.

Μ.Ι.Σ. 2025-2026

Για να διαβάσουμε μέσα από ένα αρχείο που προϋπάρχει και έχει μέσα για περιεχόμενο το,

ΑΘΗΝΑ ΙΕΚ ΔΕΛΤΑ

μπορούμε να το κάνουμε με το πρόγραμμα

```
my_b_new_arxeio = open('c:/IRIS2000/my-data.txt', 'r')

my_current_line = my_b_new_arxeio.read()

type(my_current_line)
        <class 'str'>

print(my_current_line)
        ΑΘΗΝΑ ΙΕΚ ΔΕΛΤΑ
```

Η συνάρτηση **read** διαβάζει όλα τα δεδομένα μέσα από ένα αρχείο.

Προσοχή, Χωρίς ορίσματα (αριθμό χαρακτήρων) θα διαβάσει σαν μια συμβολοσειρά όλο το περιεχόμενο του αρχείου.

Μ.Ι.Σ. 2025-2026

Μπορεί να πάρει σαν όρισμα εισόδου έναν αριθμό που θα καθορίσει πόσους χαρακτήρες θα διαβάσει, λαμβάνοντας όμως υπόψη από ποιο σημείο είχε ήδη διαβάσει με προηγούμενη **read**

(με προηγούμενο όρισμα εισόδου έναν αριθμό, έχει δηλαδή έναν δείκτη που «θυμάται»).

```
my_b_new_arxeio = open('c:/IRIS2000/my-data.txt', 'r')

my_current_line = my_b_new_arxeio.read(3)

print(my_current_line)
        ΑΘΗ

my_current_line = my_b_new_arxeio.read(2)

print(my_current_line)
        ΝΑ
```

Μ.Ι.Σ. 2025-2026

Γενικότερα τα αρχεία τα διαπερνούμε με όλες τις γνωστές αλγοριθμικές δομές επανάληψης,

Παράδειγμα διαβάσματος με διαπέραση **for** ,

Υποθέστε ότι έχουμε στον δίσκο το επόμενο αρχείο c:/IRIS2000/my-data.txt

ΑΘΗΝΑ ΙΕΚ ΔΕΛΤΑ
ΠΕΡΙΣΤΕΡΙ ΙΕΚ ΔΕΛΤΑ
ΑΓΙΑ ΠΑΡΑΣΚΕΥΗ ΙΕΚ ΔΕΛΤΑ

σε PYTHON το πρόγραμμα θα είναι

```
def   my_print_routina(my_file_name)

        my_file = open (my_file_name, 'r')

        for  my_line in my_file:

                print (my_line, end = '')

        my_file.close()

my_print_routina('c:/IRIS2000/my-data.txt')
```

Μ.Ι.Σ. 2025-2026

Για να διαβάσουμε μέσα από ένα αρχείο υπάρχουν και δυο άλλες εντολές διαβάσματος

η

readline

και η

readlines (στον πληθυντικό)

Ενώ η **read** διαβάζει όλο το αρχείο από την αρχή και μπορεί να επιστρέψει ό,τι διαβάσει μέσα σε μια μεταβλητή

η

readline διαβάζει μια-μια τις γραμμές (μέχρι και τον επόμενο χαρακτήρα αλλαγής γραμμής) **<u>και το επιστρέφει σε μια λίστα (σαν έτοιμο πίνακα-λίστα δηλαδή)</u>**

και η

readlines διαβάζει όλα τα εναπομείναντα στοιχεία (γραμμές ή απλά χαρακτήρες) μέχρι το τέλος του αρχείου **<u>και το επιστρέφει σε μια λίστα (σαν έτοιμο πίνακα-λίστα δηλαδή)</u>**

Μ.Ι.Σ. 2025-2026

Υποθέστε ότι έχουμε το επόμενο αρχείο στον δίσκο c:/IRIS2000/my-data.txt

ΑΘΗΝΑ ΙΕΚ ΔΕΛΤΑ
ΠΕΡΙΣΤΕΡΙ ΙΕΚ ΔΕΛΤΑ
ΑΓΙΑ ΠΑΡΑΣΚΕΥΗ ΙΕΚ ΔΕΛΤΑ
ΠΕΙΡΑΙΑΣ ΙΕΚ ΔΕΛΤΑ
ΚΗΦΙΣΙΑ ΙΕΚ ΔΕΛΤΑ

και το πρόγραμμα σε PYTHON,

```
my_b_new_arxeio = open('c:/IRIS2000/my-data.txt', 'r')

my_current_line = my_b_new_arxeio.readline()

print(my_current_line)
        ΑΘΗΝΑ ΙΕΚ ΔΕΛΤΑ

my_current_line = my_b_new_arxeio.readline()

print(my_current_line)
        ΠΕΡΙΣΤΕΡΙ ΙΕΚ ΔΕΛΤΑ
my_current_many_lines = my_b_new_arxeio.readlineS()

print(my_current_many_lines)
        ['ΑΓΙΑ ΠΑΡΑΣΚΕΥΗ ΙΕΚ ΔΕΛΤΑ\n', 'ΠΕΙΡΑΙΑΣ ΙΕΚ ΔΕΛΤΑ\n', 'ΚΗΦΙΣΙΑ ΙΕΚ ΔΕΛΤΑ\n']
```

Μ.Ι.Σ. 2025-2026

Είναι συνηθισμένη γενικότερη προγραμματιστική τακτική από ένα αρχείο να κάνουμε μια επεξεργασία και να δημιουργούμε ένα άλλο παράγωγο αρχείο. Έτσι και στην PYTHON, εάν υποθέσουμε ότι έχουμε το επόμενο αρχείο,

ΑΘΗΝΑ ΙΕΚ ΔΕΛΤΑ
ΠΕΡΙΣΤΕΡΙ ΙΕΚ ΔΕΛΤΑ
ΑΓΙΑ ΠΑΡΑΣΚΕΥΗ ΙΕΚ ΔΕΛΤΑ
ΠΕΙΡΑΙΑΣ ΙΕΚ ΔΕΛΤΑ
ΚΗΦΙΣΙΑ ΙΕΚ ΔΕΛΤΑ

μπορούμε να φτιάξουμε ένα δεύτερο αρχείο c:/IRIS2000/my-data-meta.txt με κάποιο κριτήριο

```
def   my_create_routina(my_file_name_a, my_file_name_b)

        my_file_a = open (my_file_name_a, 'r')
        my_file_b = open (my_file_name_b, 'w')

        while True:
                my_check_text = my_file_a.readline()

                if  my_check_text == '':
                        break
                if  my_check_text[0] == 'A':
                        continue
                my_file_b.write(my_check_text)

        my_file_a.close()
        my_file_b.close()
```

Μ.Ι.Σ. 2025-2026

my_create_routina('c:/IRIS2000/my-data-prin.txt', 'c:/IRIS2000/my-data-meta.txt')

Μ.Ι.Σ. 2025-2026

Για να γράψουμε μέσα σε ένα αρχείο μια γραμμή (το οποίο έχουμε ανοίξει για γράψιμο), θα είχαμε

```
my_b_new_line_data='ΙΕΚ ΔΕΛΤΑ ΑΘΗΝΑ'
my_b_new_arxeio. write(my_b_new_line_data)
```

με την εκτέλεση της **write,** η γλώσσα μας απαντά πόσους χαρακτήρες έγγραψε μέσα στο αρχείο

Γενικότερα όταν γράφουμε με την **write** μέσα σε ένα αρχείο της γλώσσας θα πρέπει να έχουμε υπόψη ότι το περιεχόμενο του ορίσματος της **write** πρέπει να είναι μορφής συμβολοσειράς.

Αυτό σημαίνει ότι εάν θέλουμε να γράψουμε αριθμούς πρέπει να κάνουμε τροποποίηση με την εντολή συνάρτηση **str**,

```
my_b_new_arxeio . write(str([1980, 1990, 2000, 2021]))
```

η ανωτέρω εντολή θα γράψει μέσα στο αρχείο το

[1980, 1990, 2000, 2021]

ενώ η εντολή

```
my_b_new_arxeio . write(str(2021))
```

θα γράψει το

2021

Μ.Ι.Σ. 2025-2026

Είναι συνηθισμένη γενικότερη προγραμματιστική τακτική σε ένα σειριακό αρχείο να θέλουμε να προσθέσουμε γραμμές στο τέλος τους, μετά από την τελευταία υπάρχουσα εγγραφή τους. Έτσι στην PYTHON, εάν υποθέσουμε ότι έχουμε το επόμενο περιεχόμενο μέσα σε ένα αρχείο,

ΑΘΗΝΑ ΙΕΚ ΔΕΛΤΑ
ΠΕΡΙΣΤΕΡΙ ΙΕΚ ΔΕΛΤΑ
ΑΓΙΑ ΠΑΡΑΣΚΕΥΗ ΙΕΚ ΔΕΛΤΑ
ΠΕΙΡΑΙΑΣ ΙΕΚ ΔΕΛΤΑ
ΚΗΦΙΣΙΑ ΙΕΚ ΔΕΛΤΑ

μπορούμε να προσθέσουμε εγγραφές με το πρόγραμμα,

```
my_file = open ('c:/IRIS2000/my-data.txt', 'a')

my_new_line='ΙΕΚ ΔΕΛΤΑ ΚΑΙ ΣΤΗΝ ΛΑΡΙΣΑ'

my_file.write(my_new_line)
```

με την εκτέλεση της **write,** η γλώσσα μας απαντά πόσους χαρακτήρες έγγραψε μέσα στο αρχείο

και το αρχείο μας θα περιέχει τώρα τα,

ΑΘΗΝΑ ΙΕΚ ΔΕΛΤΑ
ΠΕΡΙΣΤΕΡΙ ΙΕΚ ΔΕΛΤΑ
ΑΓΙΑ ΠΑΡΑΣΚΕΥΗ ΙΕΚ ΔΕΛΤΑ
ΠΕΙΡΑΙΑΣ ΙΕΚ ΔΕΛΤΑ
ΚΗΦΙΣΙΑ ΙΕΚ ΔΕΛΤΑ

Μ.Ι.Σ. 2025-2026

ΙΕΚ ΔΕΛΤΑ ΚΑΙ ΣΤΗΝ ΛΑΡΙΣΑ

Αντίστοιχα για να γράψουμε μέσα σε ένα αρχείο πολλές γραμμές, θα είχαμε

```
my_b_new_arxeio . writelines(my_b_new_many_lines_data)
```

Υποθέστε ότι έχουμε στον δίσκο το επόμενο αρχείο c:/IRIS2000/my-data.txt με περιεχόμενο

ΑΘΗΝΑ ΙΕΚ ΔΕΛΤΑ
ΠΕΡΙΣΤΕΡΙ ΙΕΚ ΔΕΛΤΑ
ΑΓΙΑ ΠΑΡΑΣΚΕΥΗ ΙΕΚ ΔΕΛΤΑ
ΠΕΙΡΑΙΑΣ ΙΕΚ ΔΕΛΤΑ
ΚΗΦΙΣΙΑ ΙΕΚ ΔΕΛΤΑ

και με ένα πρόγραμμα σε PYTHON, θέλουμε να δημιουργήσουμε μια λίστα, να την γεμίσουμε με 3 στοιχεία και τελικά να την κάνουμε προσθήκη στο υπάρχον αρχείο μας,

```
my_new_list=[]

my_new_line_1 = 'hello ΑΘΗΝΑ          \n'
my_new_line_2 = 'hello ΠΕΡΙΣΤΕΡΙ      \n'
my_new_line_3 = 'hello ΑΓΙΑ ΠΑΡΑΣΚΕΥΗ \n'

my_new_list.append(my_new_line_1)
my_new_list.append(my_new_line_2)
my_new_list.append(my_new_line_3)
```

Μ.Ι.Σ. 2025-2026

```
my_file = open ('c:/IRIS2000/my-data.txt', 'a')

my_file.writelineS(my_new_list)
```

οι ανωτέρω εντολές τελικά θα διαμορφώσουν το περιεχόμενο μας, στο

ΑΘΗΝΑ ΙΕΚ ΔΕΛΤΑ
ΠΕΡΙΣΤΕΡΙ ΙΕΚ ΔΕΛΤΑ
ΑΓΙΑ ΠΑΡΑΣΚΕΥΗ ΙΕΚ ΔΕΛΤΑ
ΠΕΙΡΑΙΑΣ ΙΕΚ ΔΕΛΤΑ
ΚΗΦΙΣΙΑ ΙΕΚ ΔΕΛΤΑ
hello ΑΘΗΝΑ
hello ΠΕΡΙΣΤΕΡΙ
hello ΑΓΙΑ ΠΑΡΑΣΚΕΥΗ

Μ.Ι.Σ. 2025-2026

Στα παραδείγματα που ήδη αναφέρθηκαν, χρησιμοποιήθηκε η έννοια της κάθετης διαπέρασης ώστε να προσεγγιστεί ένα αρχείο που βρίσκεται στον σκληρό δίσκο, από την πρώτη του γραμμή μέχρι και το τέλος του.

Αυτό υλοποιήθηκε, με μια «κυρίαρχη» εντολή **while** η οποία «κόλλησε», σε ένα «**True**» (ώστε να λειτουργεί ατέρμονα), μέχρι να διακοπεί από ένα **break**, με την εντολή,

```
if  my_check_text == '':
```

δηλαδή όταν δεν θα υπάρχει, τίποτα… ποια μέσα στο αρχείο και θα έχει φτάσει το τέλος του, συνεπώς δεν θα επιστρέφεται τίποτα (από το επαναλαμβανόμενο διάβασμα της **readline**).

```
my_b_file = open ('c:/IRIS2000/my-data.txt', 'r')

while True:
        my_check_text = my_b_file.readline()

        if  my_check_text == '':
                break

        if  my_check_text == 'ΑΘΗΝΑ':
                …κάνε κάτι…

        if  my_check_text == 'ΠΑΤΡΑ':
                …κάνε κάτι άλλο…

        if  my_check_text[0] == 'Α':
```

Μ.Ι.Σ. 2025-2026

```
            continue

        if  my_check_text[0] == 'Z':
            ...κάνε και εδώ κάτι άλλο...

        ...και επίσης εδώ κάνε κάτι άλλο...
        ...και συνέχιζε να κάνεις κάτι άλλο...
        ...και αυτή είναι η τελευταία εντολή, που ανήκει στο σώμα εντολών της while...

    my_b_file.close()
```

Την ανωτέρω περιγραφόμενη λειτουργία διακοπής του **while** με το **break**, μπορούμε να το πετύχουμε και με την εντολή

```
    if   len(my_check_text ) == 0:
```

η οποία, εάν αυτό που έρχεται, έχει μήκος μηδέν, τότε θα σημαίνει ότι έχει φτάσει το τέλος του αρχείου.

Όμως το τελευταίο θα λειτουργήσει σωστά μόνο εάν το αρχείο περιέχει πάντα γραμμές με περιεχόμενο (δηλαδή δεν υπάρχουν διάσπαρτες κενές γραμμές). Εάν το αρχείο περιέχει κενές γραμμές, ο κώδικας θα «νομίσει» ότι έφτασε το τέλος του αρχείου πριν αυτό να είναι πραγματικότητα.

Στο ανωτέρω πρόβλημα θα μπορούσαμε ενδεχομένως να παρέμβουμε με τη χρήση της **readlines** (στον πληθυντικό) αντί για την απλή **readline.**

Μ.Ι.Σ. 2025-2026

Επιπλέον, η γλώσσα διαθέτει και δυο άλλες ενδιαφέρουσες «εργοστασιακές» εντολές (ή έτοιμες συναρτήσεις),

την

συνάρτηση **seek**

και την

συνάρτηση **tell**

με την **tell** μπορούμε να βρούμε την θέση ενός «νοητού δείκτη» ο οποίος έχει αφεθεί μέσα στα περιεχόμενα ενός αρχείου

και στην γενικότητα της, την συντάσσουμε

```
my_b_file = open ('c:/IRIS2000/my-data.txt', 'r')
my_b_file . tell ()
```

η εντολή θα επιστρέψει έναν ακέραιο που είναι η τρέχουσα θέση του «νοητού δείκτη» μέσα το αρχείο, υπολογισμένη σε χαρακτήρες (bytes) από το σημείο αρχής του αρχείου my-data.txt

Μ.Ι.Σ. 2025-2026

Η εντολή **seek** λειτουργεί, σαν ένας δείκτης, ο οποίος «περπατά» μέσα σε ένα αρχείο και κάνει ευκολότερη στην συνέχεια την διαδικασία διαβάσματος του αρχείου,

η **seek** συντάσσεται με διάφορους τρόπους, διαθέτοντας ορίσματα, για το σκοπό αυτό

και στην γενικότητα της, την συντάσσουμε

```
my_b_file = open ('c:/IRIS2000/my-data.txt', 'r')
my_b_file . seek(W_MY_B_POSES,W_MY_B_APO)
```

Η νέα θέση του «νοητού δείκτη» υπολογίζεται προσθέτοντας το πλήθος των bytes που δηλώνουμε με το όρισμα

W_MY_B_POSES

σε σχέση όμως και με ένα σημείο αναφοράς, το οποίο το δηλώνουμε με το όρισμα

W_MY_B_APO

τα ονόματα των δυο ορισμάτων είναι ενδεικτικά.

Μ.Ι.Σ. 2025-2026

Ειδικά για το δεύτερο όρισμα (το **W_MY_B_APO**) μπορούμε να δηλώσουμε,

- το 0, για να ξεκινήσει να «περπατά» ο «νοητός δείκτης» από την αρχή του αρχείου,
- το 1, για να ξεκινήσει να «περπατά» ο «νοητός δείκτης» από το σημείο του αρχείου, που αυτός, (ο δείκτης0 βρισκόταν ήδη (από προηγούμενη ενέργεια μας)
- το 2, για να ξεκινήσει να «περπατά» ο «νοητός δείκτης» από το τέλος του αρχείου, και ανάποδα (προς την αρχή
- τίποτα, και σε αυτή την περίπτωση είναι σαν να δηλώνουμε το 1

Προσοχή, η κίνηση του «νοητού δείκτη» μέσα στα αρχεία της γλώσσας Python, δεν βαρύνεται από την δυσκολία που προκαλεί η θέση «μηδέν» των πινάκων, στις C like γλώσσες, καθώς εδώ δεν υπάρχει η θέση «μηδέν», αλλά η πρώτη θέση μέσα στο αρχείο είναι κανονικά η θέση «ένα».

Μ.Ι.Σ. 2025-2026